

Entregable Clase 1: Arquitectura Asíncrona con Python 3.12

1. Código de la Solución

A continuación se presenta el código completo estructurado siguiendo exactamente el patrón de diseño propuesto en la solución de referencia de Coderhouse (usando Semaphore como gestor de contexto en cada llamada, timeouts por tarea, y agregación mediante `asyncio.gather`):

```
import asyncio
import random
import time

# --- 1. Simulación de Llamadas: Corrutinas para cada modelo ---
# Cada una usa asyncio.sleep() y respeta el semáforo y timeout de 2 segundos.

async def gpt_4_call(call_id: int, sem: asyncio.Semaphore) -> dict:
    async with sem:
        # Latencia variable: a veces supera los 2 segundos para forzar el timeout
        latency = random.uniform(0.5, 3.0)
        print(f"[{time.strftime('%H:%M:%S')}] Task #{call_id} (GPT-4) inicia ejecución  
↳ (latencia: {latency:.2f}s)")
        try:
            async with asyncio.timeout(2.0):
                await asyncio.sleep(latency)
                print(f"[{time.strftime('%H:%M:%S')}] Task #{call_id} (GPT-4) finalizó  
↳ con éxito")
                return {"call_id": call_id, "model": "gpt-4", "ok": True}
        except TimeoutError:
            print(f"[{time.strftime('%H:%M:%S')}] Task #{call_id} (GPT-4) falló por  
↳ timeout")
            return {"call_id": call_id, "model": "gpt-4", "ok": False, "error":  
↳ "timeout"}

async def claude_3_call(call_id: int, sem: asyncio.Semaphore) -> dict:
    async with sem:
        latency = random.uniform(0.5, 1.8)
        print(f"[{time.strftime('%H:%M:%S')}] Task #{call_id} (Claude-3) inicia ejecución  
↳ (latencia: {latency:.2f}s)")
        try:
            async with asyncio.timeout(2.0):
                await asyncio.sleep(latency)
                print(f"[{time.strftime('%H:%M:%S')}] Task #{call_id} (Claude-3) finalizó  
↳ con éxito")
                return {"call_id": call_id, "model": "claude-3", "ok": True}
        except TimeoutError:
```

```

        print(f"[{time.strftime('%H:%M:%S')}] Task #{call_id} (Claude-3) falló por
        ↪ timeout")
        return {"call_id": call_id, "model": "claude-3", "ok": False, "error":
        ↪ "timeout"}

async def local_llama_call(call_id: int, sem: asyncio.Semaphore) -> dict:
    async with sem:
        latency = random.uniform(0.3, 1.0)
        print(f"[{time.strftime('%H:%M:%S')}] Task #{call_id} (LLaMA-local) inicia
        ↪ ejecución (latencia: {latency:.2f}s)")
        try:
            async with asyncio.timeout(2.0):
                await asyncio.sleep(latency)
                print(f"[{time.strftime('%H:%M:%S')}] Task #{call_id} (LLaMA-local)
                ↪ finalizó con éxito")
                return {"call_id": call_id, "model": "llama-local", "ok": True}
        except TimeoutError:
            print(f"[{time.strftime('%H:%M:%S')}] Task #{call_id} (LLaMA-local) falló por
            ↪ timeout")
            return {"call_id": call_id, "model": "llama-local", "ok": False, "error":
            ↪ "timeout"}

# --- 2. Orquestador: Dispara todas las tareas concurrentemente ---
async def orchestrate_calls() -> list[dict]:
    # El semáforo limita a un máximo de 2 ejecuciones concurrentes simultáneas
    sem = asyncio.Semaphore(2)

    # Creamos 10 llamadas alternando los modelos
    tasks = []
    for i in range(1, 11):
        if i % 3 == 1:
            tasks.append(gpt_4_call(i, sem))
        elif i % 3 == 2:
            tasks.append(claude_3_call(i, sem))
        else:
            tasks.append(local_llama_call(i, sem))

    # Lanzamos todas las tareas simultáneamente en el Event Loop
    return await asyncio.gather(*tasks, return_exceptions=True)

# --- 3. Punto de Entrada ---
async def main():
    print(f"[{time.strftime('%H:%M:%S')}] --- Iniciando Orquestador Concurrent de Modelos
    ↪ (10 llamadas, máx 2 en paralelo) ---")
    inicio = time.perf_counter()
    resultados = await orchestrate_calls()
    elapsed = time.perf_counter() - inicio

    ok = [r for r in resultados if isinstance(r, dict) and r.get("ok")]
    fallidos = [r for r in resultados if isinstance(r, dict) and not r.get("ok")]

    print("\n--- RESUMEN DE EJECUCIÓN ---")
    print(f"Procesados OK: {len(ok)}/{len(resultados)} en {elapsed:.2f}s")
    for f in fallidos:

```

```
print(f" - Falló llamada #{f['call_id']} ({f['model']}) -> {f['error']}")

if __name__ == "__main__":
    asyncio.run(main())
```

2. Logs y Verificación de la Ejecución

A continuación se adjunta la traza de ejecución generada por la terminal al ejecutar el archivo:

```
[16:25:15] --- Iniciando Orquestador Concurrent de Modelos (10 llamadas, máx 2 en
↳ paralelo) ---
[16:25:15] Task #1 (GPT-4) inicia ejecución (latencia: 2.71s)
[16:25:15] Task #2 (Claude-3) inicia ejecución (latencia: 1.08s)
[16:25:16] Task #2 (Claude-3) finalizó con éxito
[16:25:16] Task #3 (LLaMA-local) inicia ejecución (latencia: 0.70s)
[16:25:17] Task #3 (LLaMA-local) finalizó con éxito
[16:25:17] Task #4 (GPT-4) inicia ejecución (latencia: 0.61s)
[16:25:17] Task #1 (GPT-4) falló por timeout
[16:25:17] Task #5 (Claude-3) inicia ejecución (latencia: 1.31s)
[16:25:18] Task #4 (GPT-4) finalizó con éxito
[16:25:18] Task #6 (LLaMA-local) inicia ejecución (latencia: 0.64s)
[16:25:18] Task #6 (LLaMA-local) finalizó con éxito
[16:25:18] Task #7 (GPT-4) inicia ejecución (latencia: 0.99s)
[16:25:19] Task #5 (Claude-3) finalizó con éxito
[16:25:19] Task #8 (Claude-3) inicia ejecución (latencia: 0.75s)
[16:25:19] Task #7 (GPT-4) finalizó con éxito
[16:25:19] Task #9 (LLaMA-local) inicia ejecución (latencia: 0.52s)
[16:25:19] Task #8 (Claude-3) finalizó con éxito
[16:25:19] Task #10 (GPT-4) inicia ejecución (latencia: 0.84s)
[16:25:20] Task #9 (LLaMA-local) finalizó con éxito
[16:25:20] Task #10 (GPT-4) finalizó con éxito

--- RESUMEN DE EJECUCIÓN ---
Procesados OK: 9/10 en 4.90s
- Falló llamada #1 (gpt-4) -> timeout
```

Análisis del Log frente a los Criterios de Aceptación:

1. **Uso de Semáforos (`asyncio.Semaphore(2)`):** Al comenzar la ejecución, solo la Task #1 y Task #2 inician su procesamiento simultáneamente. La Task #3 permanece en cola hasta que la Task #2 finaliza a las 16:25:16. Esto demuestra que el semáforo restringe correctamente la concurrencia a un máximo de 2 tareas activas de manera simultánea.
2. **Gestión de Timeouts:** La Task #1 requería una latencia simulada de 2.71s. Dado que el bloque de timeout está configurado en 2.0s, a las 16:25:17 (exactamente 2 segundos después de iniciar), la corrutina interrumpe su ejecución, captura la excepción y libera el semáforo para permitir que la Task #5 comience. El flujo principal continúa operando con total resiliencia.
3. **No Bloqueante:** El Event Loop no se detuvo en ningún momento, ya que la duración total de la ejecución de las 10 tareas fue de únicamente 4.90s, lo cual es significativamente menor que la suma de sus latencias individuales (~12 segundos), demostrando el paralelismo cooperativo de `asyncio`.